

# Respond to Change With Constancy: Instruction-Tuning With LLM for Non-I.I.D. Network Traffic Classification

Xinjie Lin<sup>1</sup>, Gang Xiong, *Member, IEEE*, Gaopeng Gou, Wenqi Dong<sup>2</sup>, Jing Yu<sup>3</sup>, Zhen Li, and Wei Xia

**Abstract**—Encrypted traffic classification is highly challenging in network security due to the need for extracting robust features from content-agnostic traffic data. Existing approaches face critical issues: (i) Distribution drift, caused by reliance on the closed-world assumption, limits adaptability to real-world, shifting patterns; (ii) Dependence on labeled data restricts applicability where such data is scarce or unavailable. Large language models (LLMs) have demonstrated remarkable potential in offering generalizable solutions across a wide range of tasks, achieving notable success in various specialized fields. However, their effectiveness in traffic analysis remains constrained by challenges in adapting to the unique requirements of the traffic domain. In this paper, we introduce a novel traffic representation model named Encrypted Traffic Out-of-Distribution Instruction Tuning with LLM (ETool), which integrates LLMs with knowledge of traffic structures through a self-supervised instruction tuning paradigm. This framework establishes connections between textual information and traffic interactions. ETool demonstrates more robust classification performance and superior generalization in both supervised and zero-shot traffic classification tasks. Notably, it achieves significant improvements in F1 scores: APP53 (I.I.D.) to 93.19% (6.62% $\uparrow$ ) and 92.11% (4.19% $\uparrow$ ), APP53 (O.O.D.) to 74.88% (18.17% $\uparrow$ ) and 72.13% (15.15% $\uparrow$ ), and ISCX-Botnet (O.O.D.) to 95.03% (9.16% $\uparrow$ ) and 81.95% (12.08% $\uparrow$ ). Additionally, we construct NETD, a traffic dataset designed to support dynamic distributional shifts, and use it to validate ETool's effectiveness under varying distributional conditions. Furthermore, we evaluate the efficiency gains achieved through ETool's instruction tuning approach.

**Index Terms**—Encrypted traffic classification, network security, out-of-distribution generalization, large language models.

Received 22 October 2024; revised 20 May 2025; accepted 20 May 2025. Date of publication 29 May 2025; date of current version 12 June 2025. This work was supported by the National Key Research and Development Program of China under Grant 2022YFB2702404. The associate editor coordinating the review of this article and approving it for publication was Prof. Ghassan Karame. (*Corresponding author: Jing Yu.*)

Xinjie Lin is with Zhongguancun Laboratory, Beijing 100094, China, also with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100085, China (e-mail: linxj@mail.zgclab.edu.cn).

Gang Xiong, Gaopeng Gou, Wenqi Dong, Jing Yu, Zhen Li, and Wei Xia are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100085, China (e-mail: xionggang@iie.ac.cn; gougaopeng@iie.ac.cn; dongwenqi@iie.ac.cn; yujing02@iie.ac.cn; lizhen@iie.ac.cn; xiawei@iie.ac.cn).

Digital Object Identifier 10.1109/TIFS.2025.3574971

## I. INTRODUCTION

AS AN essential technology for cybersecurity and network management, traffic classification aims to identify categories of traffic from diverse applications or network services [1], [2], [3], [4], [5], [6], which has been widely used in scenarios such as security attack detection and quality of service assurance to help web content and service providers provide a more secure and high-quality web service experience for users.

In recent years, gradual full encryption of traffic has become a reality, explicit fingerprinting has been gradually failing. Different technical approaches have been proposed to address the needs of encrypted traffic analysis, including: (i) Statistical feature-based approaches [15], [40] extract statistical features and combine them with classical machine learning algorithms to cope with traffic without plaintext; (ii) Raw feature-based approaches [8], [22] on the other hand selects raw traffic features and captures complicated patterns based on deep learning algorithms; and (iii) Raw datagram-based approaches [18], [19], [20] utilize deep neural networks to learn implicit correlations between datagram bytes.

Regrettably, the validity of most encrypted network traffic analysis methods is based on the assumption that training and testing traffic are independent and identically distributed (I.I.D.), following empirical error minimization learning from the training distribution. In fact, this assumption is fragile and unrealistic in practical scenarios in the field of cybersecurity. The interaction information and patterns of web applications change over time, which makes it difficult for existing methods to ensure good performance of the test data, the most intuitive manifestations of which include version updates of web applications, and behaviors in different temporal windows [9], [10], [11]. Therefore, existing studies face the problem of probability distribution drift of traffic and category labels due to dynamic changes in network traffic, *i.e.*, new feature distributions cannot be precisely mapped to the same labels under a well-trained classification model with the old distribution [14], [31].

In response to the degraded performance of traffic classification models under Out-of-Distribution (O.O.D.) conditions, one of the most intuitive and commonly used techniques [32], [38], [39] is to periodically retrain the model to adapt to changes in traffic, as shown in Fig. 1(a). However, updating the model involves collecting labeled samples and retraining

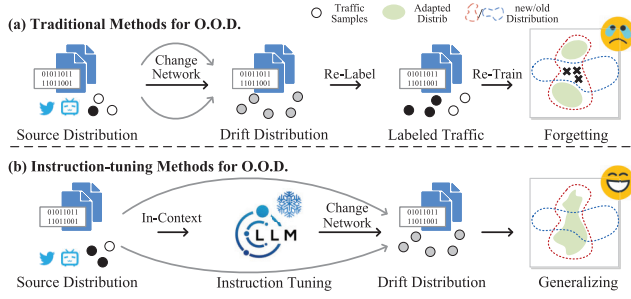


Fig. 1. The Schematic illustration of different O.O.D. identification solutions.

the classifier, consuming a lot of time and labor. Moreover, the frequent updating of applications and the forgetting of old distributions make it difficult to balance the updating effort with performance degradation [31], [32], [33]. Existing approaches are inappropriate in dealing with out-of-distribution traffic detection for two essential reasons:

- (1) **Feature Instability.** Most used traffic features, single-packet features or single-flow features inherited from packet-level information, are weakly stable under distribution drift and lack the ability to represent traffic more robustly.
- (2) **Insufficient Generalization.** Existing research frameworks are built to fit distributions under artificial experience or large-scale labeled data, while realistic networks, ranging from the complexity and variability of the application layer to the interactive changes of the transmission mechanism, launch an impact on the existing experimental assumptions.

Studies [39], [40], [41], [42] have shown that both the temporal ordering of adjacent flows and packet-level bursts contribute significantly to traffic fingerprinting. Moreover, consistent traffic burst patterns within an application, even after updates, have proven to be robust features for building stable traffic representations. Additionally, pre-training methods are gaining traction in traffic analysis. For instance, ET-BERT [12] demonstrates strong generalization across multiple tasks, offering a viable traffic representation framework, though it does not fully resolve the generalization challenge.

Recently, large language modeling has been making breakthroughs in areas such as multi-modality. Under massive knowledge and unsupervised learning tasks, LLMs are able to acquire data extrapolation and scenario transfer capabilities, and emerge as emergent capabilities in some tasks [21]. Such powerful generalization ability can be easily migrated to the target domain (Domain-LLM) [13], [16], [17] by fine-tuning the instruction form of small-scale labeled data. Stable traffic representation in conjunction with LLM motivated our idea.

To address the aforementioned challenges, we propose a novel traffic graph instruction tuning model for classifying encrypted traffic in out-of-distribution (O.O.D.) scenarios, called ETool (Encrypted Traffic Out-of-Distribution Instruction Tuning with Large Language Model). ETool focuses on designing flow interaction representations that allow LLMs to learn the underlying properties of network flows without requiring retraining for new distributions (Fig. 1(b)). First, we

introduce flow graph representations, converting flow interaction properties into learnable graph structures. By using a contrastive learning approach, we align textual representations with flow graph structures, enabling the LLM to comprehend flow characteristics. During the instruction tuning phase, the model is guided through a BURST graph matching task as a self-supervised signal. This process helps the LLM understand the underlying transport structure (BURST), enhancing its ability to capture contextual associations in traffic. In the second tuning phase, we fine-tune the LLM using traffic-specific instructions, adapting it further to the traffic identification task.

In summary, the main contributions of this paper are summarized as follows:

- I We propose an instruction tuning model, called ETool, for out-of-distribution encrypted traffic classification. The aim of this work is to align structural knowledge of the traffic domain with the generalization of LLMs, in order to enhance O.O.D. generalization for encrypted traffic.
- II We newly propose flow-specific self-supervised instruction tuning task, BURST Graph Matching, to improve the LLM's comprehension of flow interaction. Meanwhile, we introduce task-specific instruction tuning to enhance the adaptability to encrypted traffic classification.
- III We design and construct a dataset suitable for Non-I.I.D. traffic classification, named NETD. To the best of our knowledge, this is the first dynamically distributed traffic dataset that is dedicated to advancing data-supported research on O.O.D. encrypted traffic.
- IV ETool has great generalization ability and achieves a new state-of-the-art performance over 7 encrypted traffic classification datasets across independent and identically distributed and out-of-distribution scenarios, including Encrypted Application Classification, Malicious Service Classification and Encrypted Traffic Classification with Distribution Flexible, and outperforms existing works remarkably by 6.62%, 4.19%, 18.17%, 15.15%, 9.16%, 12.08% and 2.88%.

## II. PRELIMINARIES

### A. Problem Statement

An adversary can use the encrypted traffic to perform a side-channel attacks to identify whether a victim has accessed a specific set of monitored applications. A defender, on the other hand, performs intrusion detection analysis with encrypted traffic to identify whether an attacker uses a malicious program to compromise a controlled network. We assume that the attacker or defender cannot exploit the plaintext payload of the packets and define an encrypted network flow as a bidirectional sequence of packets corresponding to a unique five-tuple source IP, destination IP, source port, destination port, protocol.

The goal of out-of-distribution encrypted traffic classification is to utilize traffic data from known distributions to learn transferable traffic knowledge, in order to achieve that the mapping relationship between test data and labels stays minimally changed when the distribution is changed, and

TABLE I  
THE COMPARISON WITH THE EXISTING DATASETS OF ENCRYPTED TRAFFIC CLASSIFICATION

| Dataset                      | Year | #Flow   | Scenario                | O.O.D. | Extensible |
|------------------------------|------|---------|-------------------------|--------|------------|
| ISCX-VPN [29]                | 2016 | 2,329   | VPN/Service/Application | ✗      | ✗          |
| ISCX-Tor [28]                | 2017 | 3,021   | Tor/Application         | ✗      | ✗          |
| USTC-TFC [20]                | 2017 | 9,853   | Malware                 | ✗      | ✗          |
| Cross-Platform(iOS) [34]     | 2020 | 20,858  | Application             | ✗      | ✗          |
| Cross-Platform(Android) [34] | 2020 | 27,846  | Application             | ✗      | ✗          |
| CSTNET-TLS 1.3 [12]          | 2022 | 46,372  | Service/Application     | ✗      | ✗          |
| ISCX-Botnet [27]             | 2014 | 96,857  | Malware                 | ✓      | ✗          |
| FDAN-APP53 [41]              | 2023 | 976,000 | Application             | ✓      | ✗          |
| NETD(Ours)                   | 2024 | 3,000   | Service/Application     | ✓      | ✓          |

thus to improve the accuracy of the traffic identification task under the new distribution. Specifically, we define the out-of-distribution encrypted traffic classification as follows: Given the traffic samples in the data space  $\mathcal{X}$  and label space  $\mathcal{Y}$  as the initial data domain  $D = \{(x, y) | x \in \mathcal{X}, y \in \mathcal{Y}\} \sim P(x, y)$ , the target data domain  $D' = \{(x, y) | x \in \mathcal{X}, y \in \mathcal{Y}\} \sim P'(x, y)$  is the newly distributed traffic data obtained by sampling with a different joint probability distribution from the initial data domain, then the learning objective  $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$  is to maintain the accuracy of the label mapping in the event of a shift change in the marginal distribution of the traffic data:

$$P'(Y|X) = P(Y|X), \quad \text{if } P(X) \neq P'(X) \quad (1)$$

### B. Investigation on Existing Datasets

Datasets are an invaluable component of contemporary traffic classification research, and they have been pivotal to the tremendous progress made in the field. Not only do they serve as a reliable source of training data, but they also provide a relatively fair means of measuring and comparing the performance of competing methods. Due to the diversity of application scenarios and the development of network protocols, new traffic datasets are constantly being released to meet different research goals. However, most existing traffic datasets follow the assumption of independent and identical distribution of data, meaning that training and testing data should contain independent and identically distributed samples. In fact, we cannot decide the distribution of test data in real scenarios, and the assumption of independent identical distribution can never be strictly satisfied [25], which means that minimizing the empirical error of a model on the training data does not necessarily make it perform well on the test data.

Table I presents several representative datasets designed to meet various research objectives, covering scenarios such as VPN, malware, and mobile services. Most of these datasets were captured without fully considering the Non-I.I.D. nature of test scenarios, and only a few address distributional variations that are explicitly identified. For example, the ISCX-Botnet dataset introduces different data distributions by varying the type and volume of malicious and benign traffic between the training and testing sets. Similarly, FDAN-APP53 (APP53) accounts for both temporal and device factors

to simulate distribution shifts caused by application version changes in real-world settings. The factors contributing to distribution shifts in real-world networks are complex, and it is labor- and time-intensive to comprehensively account for and capture them. For instance, in the case of temporal distribution shifts, constructing a dataset would require data collection over multiple time periods.

In this regard, we propose NETD, an out-of-distribution encrypted traffic dataset that supports distributional dynamics adjustment while being low-cost, efficient, and usable, as it supports the exploitation of publicly available datasets. It is worth noting that no previous dataset has supported adjusting the degree of traffic distribution bias, whereas the NETD dataset supports modelling varying degrees of O.O.D. traffic in a controlled manner. Details are given in Section VIII-A.

### C. Motivation Analysis

LLMs have demonstrated remarkable performance in tasks involving rich semantics and natural language understanding, highlighting their strong generalization capabilities. When it comes to encrypted network traffic, we apply LLM-based techniques based on the following considerations:

- (1) **Feasibility of Applying LLMs to Encrypted Traffic Classification.** Although encrypted traffic lacks rich semantic information, the use of LLMs in encrypted traffic analysis is gaining momentum. Several studies [12], [13] have shown that LLM architectures (particularly leveraging pre-training) significantly enhance generalization in traffic classification tasks. By representing traffic features in a sequential format, LLMs can be naturally integrated to improve model generalization.
- (2) **Effectiveness of Structured Traffic Graph Representations.** Structured graph representations of traffic have been proven effective in capturing stable interaction patterns between flows [41]. To further address the challenge of distributional shift in traffic data, we propose leveraging traffic graphs as an input representation for fine-tuning LLMs. Our experiments confirm that incorporating graph-based representations provides measurable gains in performance under distribution shift conditions.

### (3) **Feasibility of LLMs Learning from Structured Data.**

While LLMs are inherently designed for text, recent research [49] has demonstrated their ability to generalize across structured data formats. This inspired us to explore the integration of LLMs with graph-structured representations of network traffic, aiming to harness their transferability and generalization in non-textual domains.

### (4) **Advantages of the LLM Architecture.** We evaluated LLM-based approaches using traffic sequence representations [12], [24] alongside traditional AI techniques. Notably, most of these methods do not explicitly address the issue of distribution shift. Other existing approaches [31] rely on access to unknown traffic in advance, which deviates from the strict requirements of true O.O.D. detection. In contrast, the LLM architecture exhibits stronger generalization capabilities, including zero-shot learning and representational transfer, which motivated our investigation.

## III. RELATED WORK

In this section, we provide an overview of encrypted traffic classification methods that have been proposed, including statistical feature-based methods and deep learning-based methods, as well as pre-training and instruction tuning. Fingerprint construction represented by deep packet inspection (DPI) is no longer applicable and will not be discussed.

### A. Encrypted Traffic Classification

1) *Statistical Methods:* To efficiently analyze complex traffic, most studies of encrypted traffic utilize statistical features of traffic independently of traffic encryption. CUMUL [15] selects 104-dimensional statistical features by accuracy evaluation and then utilizes them as input to a support vector machine to identify website traffic. AppScanner [40] uses statistical features of packet size to train a random forest classifier. While ML-based methods combined with statistical features can analyze complex traffic, they rely on expert-designed statistical features, which makes it difficult to design generic statistical features to adapt to the large number of applications and websites that are constantly changing [36].

2) *Deep Learning Models:* Encrypted traffic classification using supervised deep learning in conjunction with raw features or raw datagram has become a popular approach to automatically extract distinguishing features rather than relying on manual design. FS-Net [22] uses recurrent neural networks (RNNs) to automatically extract representations from the original packet size sequence of encrypted traffic, while Deeppacket [19] and TSCRNN [18] are representing the original payload. Traffic Interaction Graph [7] models flow interactions as graphs and learns flow associations based on graph representations, providing better traffic identification ability. However, such methods rely on a large amount of supervised data to capture the effective features and thus learn biased representations in a small range of data.

### B. Pre-Training and Instruction Tuning

Pre-training techniques learn unbiased data representations from large amounts of unlabeled data through self-supervised learning, which not only significantly reduces the appetite for labeled training data, but also further improves performance in downstream tasks. In encrypted traffic classification, pre-training models are applied as emerging architectures to improve the generalization of traffic classification. PERT [24] applies the pre-training model to migrate ALBERT to encrypted traffic classification and achieves performance improvement in VPN scenarios. ET-BERT [12] proposes pre-training tasks that are more suitable for traffic datagram representation and achieves performance improvement in traffic classification under multiple tasks, which demonstrates the powerful generalization of the pre-training model for encrypted traffic classification. In addition, MT-FlowFormer [46] and Flow-MAE [47] utilize the pre-training model to capture flow correlations from a visual perspective and improve flow identification performance. Pre-training techniques demonstrate power in traditional traffic classification, but these studies fall short of the desired goal by not considering attempts to solve the problem of classifying Non-I.I.D. encrypted traffic.

Prompting [37], a technique that uses task-specific prompts to guide pre-trained models, reducing the need for fine-tuning or large amounts of labeled data. Recently, prompt learning has demonstrated its effectiveness in generalized transfer in natural language processing, computer vision, and network service optimization tasks [43], [45]. The instruction tuning paradigm, integrating the pre-train and fine-tuning framework with prompt learning, enhances generalization capabilities in transfer learning by enabling effective task adaptation with few or even zero samples [26], [48]. In the context of encrypted traffic classification, this area of research remains largely unexplored. We propose a generic traffic representation based on domain-specific traffic knowledge, taking into account the unique characteristics of traffic modalities. Additionally, we design two instruction-tuning tasks to ensure the generalized transferability of traffic representations.

## IV. OVERVIEW OF ETool

In this section, we present the design of ETool. Typically, encrypted traffic analysis focuses on extracting multi-dimensional features from a single data flow and examines the flow pattern under the assumption of I.I.D. data. However, this assumption is often fragile and unrealistic in real-world scenarios. Single-flow patterns are more susceptible to performance degradation due to distributional bias compared to multi-flow interaction patterns.

To this end, our goal is to learn generic interaction correlation patterns for encrypted traffic and achieve better netflow classification in scenarios with different distributional variations. Thus, in this paper, we propose ETool, an out-of-distribution encrypted traffic classification framework based on a generalized pre-trained large model, to tackle the out-of-distribution recognition problem in encrypted traffic in the network domain. As shown in Fig. 2, the ETool framework



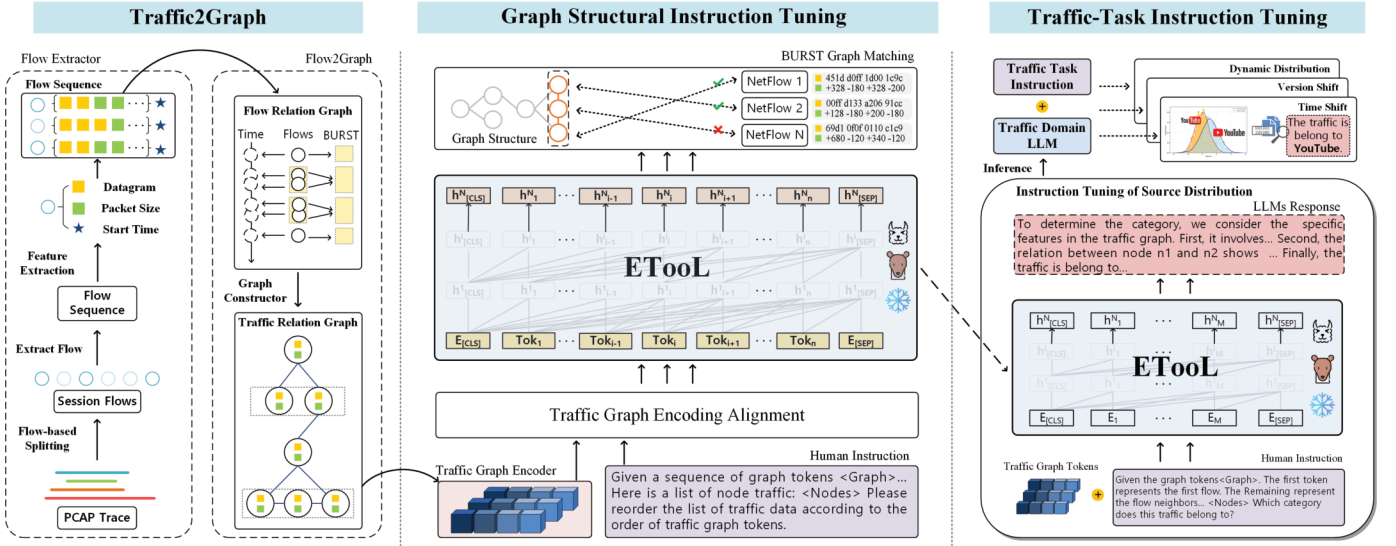


Fig. 2. Overview of ETool framework.

consists of a two-stage fine-tuning and contains three core components, *i.e.*, traffic interaction graph structure representation, graph structure instruction tuning, and traffic task instruction tuning.

**Traffic2Graph:** Drawing inspiration from FRG features [41] that cope with ambiguous flows and concept drifts, we propose to utilize flow interactions incorporating multi-granularity features as a generic pattern for constructing flow representations. Network Traffic Relation Graph (TRG) is constructed based on the correlation topology between different flows, and the graph contains flow features at different granularities. Specifically, the TRG consists of multiple network flows at adjacent timing, where each node represents a network flow, and each node contains a Raw Datagram (RD) sequence and a Packet Length (PL) sequence. The relationship between nodes represented by different network flows, *i.e.*, correlation edges, consists of *adjacency edges* and *burst edges*, which represent the interaction between different flows.

**Graph Structural Instruction Tuning:** This phase proposes a well-designed flow-graph alignment module and a flow-graph structure instruction tuning paradigm for helping LLMs capture and learn flow associations, thus alleviating problems such as the difficulty of existing LLMs in understanding flow feature information and flow-graph structure. In particular, the flow-graph alignment module aims to align the flow features and topological relationship graphs in the encoding space, based on which natural language instruction data containing flow feature information is designed for self-supervised tuning, leading to better understanding of flow graph structure knowledge.

**Traffic-Task Instruction Tuning:** In order to help the large language model adapt to the out-of-distribution traffic identification task, this phase proposes the traffic task instruction tuning module to design the instruction data for traffic classification based on the knowledge of the traffic domain structure obtained by the large language model.

We realize the learning of network interactions and contextual associations by constructing correlation graphs to represent the interaction characteristics of multi-flow, and allowing the pre-trained models to recognize the flow correlation patterns through self-supervised learning. Meanwhile, with the inference and understanding ability of the pre-trained model, we realize the generalized out-of-distribution flow identification under the traffic task instruction tuning.

## V. TRAFFIC2GRAPH

In real networks, multiple network flows are often established within a short timeframe to enhance the application's response rate and improve user experience. This results in several communication flows being created and transmitting messages simultaneously, which can be observed through passive network traffic capture.

This phenomenon is referred to as BURST, a key concept widely used in recent years for traffic feature mining and representation. Specifically, *packet-level BURST* is defined as a sequence of consecutive packets whose arrival intervals are within a small time threshold. Several studies have demonstrated that this traffic structure is effective for analyzing encrypted network traffic. Similarly, *flow-level BURST* refers to network flows established within a short time window, which helps explore the correlation properties between flows. While packet-level BURST captures the traffic characteristics of multiple packets serving the same resource request or response, flow-level BURST reflects the collaboration between multiple flows serving the same network function.

To further capture the interactions between flows in raw traffic, we propose the Traffic2Graph module to construct a discriminative and generic traffic characterisation with the variability that exists in the flow-level BURST features of different web applications. The module consists of two processes: (1) Flow Extractor extracts datagrams and packet sizes from each input network flow to merge flow feature information of different dimensions. (2) Flow2Graph further constructs the

flows with fused features into a graph structure according to timing and connectivity relationships to better represent the correlation information among different flows in BURST.

Web application developers often implement similar functionalities, but their differing interpretations of business models and development practices can result in distinct network flow collaboration patterns at the network traffic level. This section examines the interaction between network flows, focusing on two key relationships: adjacency and bursting. Adjacency describes the connectivity between adjacent network flows and, in this context, is extended to represent the through-connectivity between neighboring BURST structures. The bursting relationship, on the other hand, refers to the connectivity between flows within the same flow-level BURST structure. These multi-dimensional correlations capture the temporal and sequential relationships between network flows and the collaborative construction of BURST structures, which work together to enable network service functionality.

#### A. Flow Extractor

The multi-dimensional fusion of traffic information helps to adapt to the needs of more traffic identification scenario tasks, and we focus on datagrams and packet sizes that are widely used and have effects: datagram and packet size. In particular, to adapt to flow data in instruction fine-tuning, we expand the token representation of LLM according to CETP [30].

- (1) Sequence of Datagrams. We extract 128 bytes from the datagrams and construct traffic representation units that contain more information to enable the pre-training phase to obtain richer contextual information. Therefore, the hexadecimal bit sequence in the original traffic datagram is double-byte split and encoded as a sequence of byte pairs, where the representation space of each unit ranges from 0 to 65535, *e.g.*, the {ee08bf56...} would be represented as {ee08, bf56,...}.
- (2) Directed Packet Size Sequence. The construction of the packet size sequence follows the conventional way of flow statistical characterisation, where the packet size is extracted while preserving the communication direction information of the encrypted flows, where + indicates that the packet is sent from the client to the server, and - indicates that the packet is returned from the server to the client. For example, a sequence of directed packet sizes for a bi-directional flow can be represented as {+128, -74, -1020, +378...}.

#### B. Flow2Graph

Given all network flows  $S$  generated by a client using a certain web application during a certain period of time, and a graph structure is constructed for these flows. Taking advantage of the property that graph data structures can express rich node information and relationships between disjoint nodes, the traffic relation graph TRG( $G = (V, E)$ ) is used to express adjacency and bursting relationships between different network flows. The specific construction process of the traffic relation graph is shown in Algorithm 1.

---

#### Algorithm 1 Construction of Traffic Relation Graphs

---

**Input:**  $S = \{f_1, f_2, \dots, f_n\}$ : Network traffic data collected over time;  $\gamma$ : Interval threshold for determining the flow level BURST;

**Output:**  $G = (V, E)$ : The traffic relation graph  $G$  containing nodes  $V$  and edges  $E$ ;

```

1:  $V = \{\}, E = \{\}, BURST = [], BURST_{last} = []$ 
2: Sort the network flows in  $S$  by starting timestamps
3: Each flow is added as a node to  $V$ 
4: for each  $f \in S$  do
5:   if  $BURST \neq NULL$  then
6:     if  $|f_{start\_time} - BURST_{last\_start\_time}| \leq \gamma$  then
7:       current flow  $f$  is added to BURST
8:     else
9:       for  $i \in range(BURST_{size})$  do
10:         $E.insert(BURST[i], BURST[i + 1])$ 
11:      end for
12:      if  $BURST_{last} \neq NULL$  then
13:         $E.insert(BURST_{last}[-1], BURST[0])$ 
14:         $E.insert(BURST_{last}[-1], BURST[-1])$ 
15:      end if
16:       $BURST_{last} = BURST$ 
17:       $BURST = []$ 
18:    end if
19:  end if
20: end for
21: return  $G = (V, E)$ 

```

---

In accordance with the adjacency and bursting edges between flows in the temporal relationship, and the feature information representation in each flow, we construct the nodes and edges in the traffic relation graph as follows:

- (1) Nodes  $V$  in TRG. Each network flow constitutes a node in the TRG, where each node consists of flow features including datagram sequences, packet size sequences, packet message type sequences, packet time interval sequences, and so on. Since the value of the start timestamp of a network flow is dynamically variable, we are mainly concerned with the directed packet size sequence and datagram sequence of each network flow.
- (2) Edges  $E$  in TRG. The TRG contains two types of association edges, the first of which is adjacency edge for connecting flows within different BURST structures. By capturing the bursting relation, the flows in the set  $S$  can be divided into different BURST structures, after which the neighbour relation is formed by connecting the last flow of the previous flow level BURST to the first and last flows of the next BURST. And the second one is the burst edge, which is applied to connect concurrent network flows within the same BURST structure. The flow level BURST is divided according to whether the start timestamp of the network flow is within a small temporal neighbourhood  $\gamma$ .

## VI. GRAPH STRUCTURAL INSTRUCTION TUNING

To enhance the understanding of flow graph structural information with LLMs, ETool aligns the flow graph structural

encoding with the natural language space. This alignment is intended to enable the language model to leverage the inherent language comprehension capabilities for effective understanding of flow features and through-connection relationships. Towards this goal, we design a flow graph encoding alignment module that aims to preserve the flow graph structural context information during instruction tuning of the large language model, thus effectively correlating the flow understanding with the topological structure relationships in the graph.

#### A. Traffic Graph Encoding Alignment

Inspired by cross-modal alignment studies such as CLIP, we integrate traffic features into the graph structure encoding process in the form of contrastive learning to align and fuse the traffic graph structure and traffic information representation.

Specifically, a graph neural network encoder with pre-training parameters is integrated into the ETool framework and enabled to correspond to the graph representation and the flow representation encoding through the contrastive learning approach. Assuming that the flow graph is represented as  $G(V, E, A, X)$ , the flow feature information of the  $n$ th node corresponding to the flow is represented as  $C = \{c_i \in \mathbb{R}^{l_i \times d}, 1 \leq i \leq N\}$ , where  $l_i$  denotes the length of the input of the  $i_{th}$  node and  $N$  denotes the number of nodes.

The encoded flow graph structure and flow feature representations are obtained by any graph representation encoder  $f_g$  (e.g., Graph Transformer) and flow representation encoder  $f_n$  (e.g., ET-BERT) as follows:

$$H = f_g(G), N = f_n(C) \quad (2)$$

where  $V, E, A, X$  as inputs to the flow graph denote the node encoding, associated edges, graph adjacency matrix, and node features, respectively.  $H$  denotes the structure-level graph encoding representation generated by the graph neural network encoding, and  $N$  is the encoded representation of the flow features associated with the nodes.

The traffic-graph alignment process for different dimensions through comparative learning is conducted as follows:

$$\mathfrak{I}_i = (g_i^1(\text{norm}(H)) \cdot g_i^2(\text{norm}(N))^\top) \cdot \exp(\tau) \quad (3)$$

$$L = \sum_i \frac{1}{2} \lambda_i (\text{CE}(\mathfrak{I}_i, y) + \text{CE}(\mathfrak{I}_i^\top, y)) \quad (4)$$

where  $y = (0, 1, \dots, -1)^\top$  as the contrastive learning target denotes the alignment label,  $\mathfrak{I}_i$  denotes the similarity measure during contrastive learning,  $g_i$  denotes the encoder of different information,  $\tau$  denotes the temperature coefficient,  $\lambda$  denotes the weight coefficient of different difficulty sample pairs, and CE denotes the cross-entropy loss function.

#### B. Burst Graph Matching

The encoding alignment enables the inclusion of flow features in the instruction cues to be understood in association with the flow graph structure. In order to further align the linguistic comprehension of the large language model with the graph learning task, we utilise a pre-trained instruction tuning paradigm to enhance the adaptability of the large language

model for specific traffic learning tasks, enabling the large language model to generate more accurate and contextually appropriate results for the traffic graph structure data.

Despite the strong generalised contextual understanding of the large language model, it is lacking in the understanding of network communication behaviours as well as traffic features. Meanwhile, the construction of traffic understanding capability is independent of a specific traffic identification task, and we use self-supervised instruction tuning to inject the knowledge of traffic graph structure into the large language model so as to effectively understand the contextual information in the traffic. Specifically, we design the interactive structure-aware flow graph matching task in the self-supervised instruction tuning approach, which allows the use of unlabelled encrypted traffic data to generate the representations of the flow graph structure as part of the instructions for the tuning of the large language model, where the flow graph structure will be used as a self-supervised signalling unit to instruct the large language model to distinguish between the different flow graph nodes using both the natural language and the flow sequence.

*Instruction Design:* The traffic graph matching task is guided by three core components: (i) traffic graph, (ii) problem instruction, (iii) ETool response. Each node within the traffic graph is designated as a central node, and an h-hop random neighbor sampling strategy is employed to extract the subgraph structure from the input traffic graph. The input provided to the large language model is a combination of natural language descriptions and traffic-specific features. For this task, the instructions consist of a <graph> indicator unit, disrupted BURST traffic features, and a textual problem description. The objective is to align each flow, represented by a traffic graph node, with its corresponding traffic feature. Achieving this alignment necessitates reordering the disordered BURST traffic feature representations by understanding the topological relationships between graph nodes. Through this process, the model correlates the structural representation of the traffic graph with its associated traffic features, thereby improving its capacity to infer and comprehend network traffic behavior.

*Tuning Strategy:* Through the lightweight alignment projection, we keep the parameters of both the LLM and the graph neural network encoder frozen during tuning, optimizing only the parameters within the projection layer. Specifically, we freeze all components of the LLM backbone, including attention blocks, token embeddings, and layer normalization layers, as well as every layer within the pre-trained flow-graph encoder. After fine-tuning, the projection layer effectively maps encoded flow-graph representations to corresponding node representations, enabling the LLM to align these node representations with various node-level feature semantics. By using a projector (e.g., a linear mapping module), the model establishes a correspondence between graph node representations and flow feature instruction representations. The indicator unit <graph> embedded in the natural language instructions is replaced with the aligned flow graph node representations, formatted as {<graph\_begin>, <graph\_token><sub>1</sub>, ..., <graph\_token><sub>n</sub>, <graph\_end>}. This incorporation of flow graph structure into the instructions allows the large language model to process them. Since the flow graph

TABLE II

THE STATISTICAL INFORMATION OF DATASETS. THE ACTUAL NUMBER OF DATASETS USED FOR CLASSIFICATION AFTER PROCESSING. THE CATEGORIES AND SAMPLE SIZE OF NETD ARE DETERMINED BY THE ACTUAL DATASET USED

| Tasks               | Description of Task                                                   | Dataset                          | #Flow        | #Label | Collection |
|---------------------|-----------------------------------------------------------------------|----------------------------------|--------------|--------|------------|
| EAC-T               | Encrypted Application Classification with the Same Time Distribution  | APP53-TIME [41]                  | 93,479       | 28     | Public     |
| EAC-V               | Encrypted Application Classification for the Same Application Version | APP53-VERSION [41]               | 104,697      | 25     | Public     |
| EAC $\Rightarrow$ T | Encrypted Application Classification with Time Shift                  | APP53 $\Rightarrow$ TIME [41]    | 93,479       | 28     | Public     |
| EAC $\Rightarrow$ V | Encrypted Application Classification with Version Shift               | APP53 $\Rightarrow$ VERSION [41] | 104,697      | 25     | Public     |
| MSC $\Rightarrow$ T | Malicious Service Classification with Type Shift                      | ISCX-Botnet [27]                 | 8,000(5,578) | 2(8)   | Public     |
| ETC $\Rightarrow$ F | Encrypted Traffic Classification with Distribution Flexible           | NETD                             | -            | -      | Private    |

matching process is self-supervised, it efficiently leverages large amounts of unlabeled flow graph data from various traffic scenarios, improving the generalization capabilities of the learned projectors.

## VII. TRAFFIC-TASK INSTRUCTION TUNING

After completing self-supervised instruction tuning, we develop task-specific instruction tuning methods tailored to encrypted traffic classification. This process involves customizing the inference behavior of the LLM to meet the specific constraints and requirements of the classification task. By fine-tuning the LLM with task-specific instructions, the model is guided to generate responses that are more appropriate for traffic learning. The traffic task instruction tuning further enhances the model's adaptability in handling encrypted traffic identification tasks across varying distributions.

In the traffic task instruction tuning, the instruction template consists of three parts. The traffic graph information includes multiple traffic samples collected over time, supporting the construction of sub-graphs. For the traffic classification task  $\phi$ , we model the traffic representation using the training pair  $(X, y)$ . During this phase we continue to keep the full LLM backbone and the flow-graph encoder frozen, updating only the structure-aware projector inherited from BURST Graph Matching together with a lightweight task-specific classification head. Then ETool leverages the parameters of the structure-aware projector, trained in the first phase as the initial state  $\theta$ , and fine-tunes it to predict the traffic label  $y$ .

$$y = ETool(X|(\theta; \phi)) \quad (5)$$

After completing the dual-stage instruction tuning, which includes freezing specific model parameters, the large language model's ability to understand and infer the structure of traffic graphs is significantly improved. This approach enables the model to efficiently handle a wide range of tasks related to traffic graph analysis.

## VIII. EVALUATION

In this section, we perform six different encrypted traffic classification scenario tasks (Section VIII-A) to demonstrate that ETool has better generalisation and effectiveness in the out-of-distribution traffic identification task, as well as to show

that the ETool model can still be adapted to the old distributed encrypted traffic identification task. We then compare our model with 6 approaches (Section VIII-B) and perform an ablation analysis of the key components of the model (Section VIII-C). We further provide an analysis of the effectiveness of ETool for traffic identification in a dynamic distribution offset scenario (Section VIII-D), an evaluation of the efficiency of the model (Section VIII-E), as well as the analysis of hyper-parameter selection (Section VIII-F).

### A. Experimental Settings

1) *Datasets Descriptions*: To evaluate the effectiveness and generalization of ETool, we conduct experiments across five encrypted traffic classification tasks on four public datasets and one newly proposed dataset. The tasks and the corresponding datasets are shown in Table II.

We conduct I.I.D. and O.O.D. experiments in the publicly available dataset APP53 [41], which contains the 53 web apps with the largest user sizes selected from the Google Apps Marketplace, and collect data from these apps across time and versions on different devices by volunteers. However, since this dataset only exposes the encrypted traffic dataset of apps collected on Xiaomi 5Plus devices at different times and with different versions, the following task will be set up around that: extracting only apps from APP53 that have undergone version changes and time changes, and setting them up as Independent Identically Distributed and Non-Independently Identically Distributed experiments.

Although the aforementioned APP53 dataset has some scenarios for O.O.D. traffic identification, existing publicly available O.O.D. datasets that can be used for evaluation are still scarce. At the same time, most of the publicly available datasets follow the experimental assumption of independent and identical distributions without considering the impact of distributional variations, which lacks explicit support for the evaluation of O.O.D. traffic identification. Datasets for simulating different O.O.D. traffic scenarios to better support traffic classification studies are still vacant.

In order to more efficiently support and promote the research of O.O.D. encrypted traffic analysis, we design and construct NETD (Dynamic Non-I.I.D. Encrypted Traffic Dataset), an O.O.D. encrypted traffic dataset that supports dynamically adjustable distributions, within the existing



publicly available conventional encrypted traffic dataset, ISCX-VPN [29]. We construct NETD by exploiting the intrinsic variability of target concepts and background contexts in the traffic data to achieve distributional shifts, *i.e.*, to simulate the distributional changes brought about by the development of objective factors such as time. By treating any network behaviour as a contextual principal component and other network behaviours as secondary components in the traffic data, the degree of flexibility in controlling the distributional shift can be achieved by adjusting the proportional deviation of the principal and secondary components in the target traffic task.

Given a feature extractor  $G$  as well as a category  $C$ , we introduce the Non-I.I.D. Index (NI) [35] used to evaluate the impact of distributional shift on the dataset as follows:

$$NI(C) = \left\| \frac{G(X_{Train}^C) - G(X_{Test}^C)}{\sigma(G(X^C))} \right\|_2 \quad (6)$$

where  $X$  denotes the full data set and  $X_C = X_{Train}^C \cup X_{Test}^C$ . The  $\overline{(\cdot)}$  represents the first-order moments, meaning that the expected or mean value is calculated for the representation of the training set or the test set, which is able to systematically portray the probability distribution of the data set. The  $\sigma(\cdot)$  represents the standard deviation, which is used to normalize the dimension of the representation. The  $\|\cdot\|_2$  represents the L2 norm, which is able to measure the degree of difference in the distribution between the train and test sets.

To verify the prevalence of Non-I.I.D. in existing datasets, we use the ET-BERT model as a feature extractor and test it on the widely used I.I.D. dataset ISCX-VPN. Figure 3 illustrates the impact of distributional shifts on traffic identification, where traffic classes with more severe distributional shifts produce correspondingly larger identification errors in testing, and the strict independent identity distribution is difficult to satisfy, *i.e.*, few classes are able to achieve a NI value of zero.

2) *Downstream Tasks*: In accordance with Table II, we present six encrypted traffic classification tasks:

Task 1: **Encrypted Application Classification with the Same Time Distribution (EAC-T)** aims to set up and classify the identification of application traffic that is under the same time.

Task 2: **Encrypted Application Classification for the Same Application Version (EAC-V)** aims to classify application traffic collected from the same version of the application.

Task 3: **Encrypted Application Classification with Time Shift (EAC  $\Rightarrow$  T)** aims to classify application traffic based on time span (one month interval). Task 1 will be fine-tuned under this task and tested in the form of zero-shot on traffic collected at different times.

Task 4: **Encrypted Application Classification with Version Shift (EAC  $\Rightarrow$  V)** aims to categorise application traffic based on version span (version update). Task 2 will be fine-tuned under this task and tested in the form of zero-shot on traffic data collected from another application version.

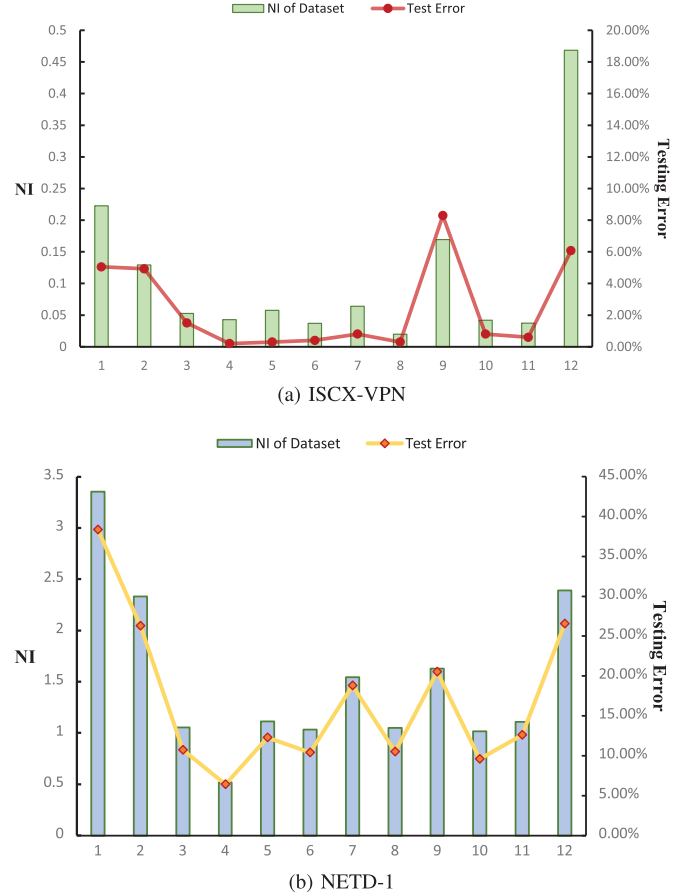


Fig. 3. Comparison of the index of distribution shift and testing error.

Task 5: **Malicious Service Classification with Type Shift (MSC  $\Rightarrow$  T)** aims to classify the botnet traffic with type shifts. In this task, benign traffic is one class, while botnet traffic consists of 16 types, of which the training set contains only a portion of seven of them. In addition to the default binary classification, we add a multi-classification scenario.

Task 6: **Encrypted Traffic Classification with Distribution Flexible (ETC  $\Rightarrow$  F)** aims to classify the encrypted traffic under different distributional variations. In this task, twelve web services are grouped into four different distributional datasets through different shifts in the context.

**Notes.** In our analysis of the APP53, we identified inconsistencies between the application categories linked to the shift factors and the descriptions in the original work, along with difficulties in obtaining fully accurate labels. In APP53, version changes involve 25 application categories, not 22 as originally described, and labeling inconsistencies contributed to lower test accuracy. Additionally, in ISCX-Botnet (Multi-Class), only 5 classes could be mapped to known labels. Thus, we supplemented the remaining two categories based on the original combined datasets. As a result, we conducted the experiments in this paper based on the actual dataset acquisition to ensure accurate evaluation results. The category distribution can be seen in Table III.

TABLE III  
CATEGORY DISTRIBUTION OF THE DATASETS

| Dataset                        | #Label | Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APP53-TIME<br>APP53⇒TIME       | 28     | air.ITVMobilePlayer, bbc.mobile.weather, cn.cntv, com.amazon.kindle, com.booking, com.cnn.mobile.android.phone, com.dropbox.android, com.flipkart.android, com.google.android.gm, com.groupon, com.gumtree.android, com.iconology.comics, com.imdb.mobile, com.imo.android.imoim, com.joelapenna.foursquared, com.lenovo.anyshare.gps, com.pinterest, com.reddit.frontpage, com.shazam.android, com.shpock.android, com.skype.raider, com.soundcloud.android, com.ted.android, com.tripadvisor.tripadvisor, info.androidz.horoscope, kik.android, sg.bigo.live, tv.twitch.android.app |
| APP53-VERSION<br>APP53⇒VERSION | 25     | com.amazon.mShop.android.shopping, com.badoo.mobile, com.contextlogic.wish, com.facebook.katana, com.google.android.youtube, com.guardian, com.ideashower.readitlater.pro, com.instagram.android, com.jingdong.app.mall, com.kakao.talk, com.nytimes.android, com.particlenews.newsbreak, com.sina.weibo, com.snapchat.android, com.spotify.music, com.ss.android.ugc.aweme, com.taobao.taobao, com.tencent.qqlive18n, com.twitter.android, com.vidio.android, com.vkontakte.android, com.xunmeng.pinduoduo, flipboard.app, ru.ideast.championat, tv.danmaku.bili                     |
| ISCX-Botnet                    | 2(8)   | Benign and Malicious (Neris, Rbot, Virut, NSIS, SMTP Spam, Zeus, Zeus Control)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

Echoing these tasks, we implement experiments to validate the effectiveness of our framework in a variety of settings and address key research questions:

- **RQ1:** How does the proposed ETool framework perform in both supervised and zero-shot settings for traffic classification? (Section VIII-B)
- **RQ2:** What is the contribution of various key components in the proposed ETool framework to its overall performance? (Section VIII-C)
- **RQ3:** What is the generalization ability of our model in handling dynamic distribution shift? (Section VIII-D)
- **RQ4:** How efficient is the ETool framework? (Section VIII-E)
- **RQ5:** What extent does hyper-parameter selection affect the results? (Section VIII-F)

3) *Comparison Methods:* The state-of-the-art (SOTA) methods used by application fingerprinting are summarized as comparison approaches, including (1) statistical feature methods: AppScanner [40] and CUMUL [15]; (2) deep learning methods: Deep Fingerprinting (DF) [8], FS-Net [22] and GraphDApp [7]; (3) pre-training methods: PERT [24] and ET-BERT [12]. These methods are selected because they represent a subset of different technical approaches that support

flow-level traffic detection. They are representative, widely adopted, and commonly used as baselines in comparative studies.

4) *Evaluation Metrics:* For each experiment, we evaluate the methods by four typical metrics, including Accuracy ( $AC = (TP+TN)/(TP+TN+FP+FN)$ ), Precision ( $PR = TP/(TP+FP)$ ), Recall ( $RC = TP/(TP+FN)$ ), and F1-Score ( $F1 = (2 * PR * RC)/(PR + RC)$ ). Macro Average [23] is used to avoid biased results due to imbalance between multiple categories of data by calculating the mean value of PR, RC and F1 of each category.

$$\begin{aligned} \text{Macro-Precision} &= \frac{1}{N} \sum_{i=1}^N \text{Precision}_i, \\ \text{Macro-Recall} &= \frac{1}{N} \sum_{i=1}^N \text{Recall}_i, \\ \text{Macro-F1} &= \frac{1}{N} \sum_{i=1}^N F1_i. \end{aligned} \quad (7)$$

5) *Implementation:* We employ Vicuna-7B-v1.5 as the base model for our approach. Unlike traditional Transformers, this model is optimized by several key modifications: replacing LayerNorm with RMSNorm, Multi-Head Attention with Grouped-Query Attention, Positional Encoding with Rotary Position Embedding, and ReLU with SwiGLU as the activation function. The architecture consists of 32 decoder layers, each with 32 self-attention heads, and the dimensions of the q, k, and v vectors in the attention module are set to 128. ETool is set with the learning rate of  $2 \times e^{-3}$ , a warmup ratio of  $3 \times e^{-2}$ , the training epoch of 3, the batch size of 2 and the maximum input length of the LLM to 2,048. We set the BURST time threshold to a value of 1s. All the experiments are implemented with Pytorch 2.1.0, conducted with NVIDIA Tesla A800 GPUs with 80 GB.

#### B. Overall Performance Comparison (RQ1)

We conduct experiments on the traffic classification tasks, evaluating both supervised and zero-shot scenarios. The overall performance is presented in Table IV. Our ETool consistently outperforms various state-of-the-art baselines in both supervised and zero-shot scenarios.

1) *General Encrypted Traffic Classification in I.I.D:* We first discuss the performance between our proposed model and the existing methods in classifying apps in ideal experimental setting. The experiments in this section also play the role of baseline to indict the effect of ambiguous traffic and concepts drift in the following sections.

Our proposed ETool achieves an average performance improvement of 5.41%, 7.49%, and 19.87% on F1 compared to different representative traffic classification baselines (ET-BERT, FS-Net, and AppScanner) in I.I.D. scenarios.

In the EAC-TIME task, ETool achieves performance improvements of 6.62%, 7.63%, and 20.92% in F1 score compared to ET-BERT, FS-Net, and AppScanner, respectively. The APP53 dataset's homogeneous flow interference hampers traditional traffic feature construction methods (such

TABLE IV  
COMPARISON RESULTS ON APP53 IN I.I.D. AND O.O.D. SETTING

| Dataset      | APP53-TIME   |              |              |              | APP53-VERSION |              |              |              | APP53 $\Rightarrow$ TIME |              |              |              | APP53 $\Rightarrow$ VERSION |              |              |              |
|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|--------------|--------------------------|--------------|--------------|--------------|-----------------------------|--------------|--------------|--------------|
| Method       | AC%          | PR%          | RC%          | F1%          | AC%           | PR%          | RC%          | F1%          | AC%                      | PR%          | RC%          | F1%          | AC%                         | PR%          | RC%          | F1%          |
| AppScanner   | 72.65        | 72.90        | 72.65        | 72.27        | 73.08         | 73.53        | 73.53        | 73.29        | 43.87                    | 46.02        | 43.87        | 44.03        | 35.39                       | 35.85        | 35.69        | 35.00        |
| CUMUL        | 60.13        | 59.93        | 60.38        | 59.74        | 61.00         | 60.18        | 61.04        | 60.22        | 37.78                    | 37.91        | 37.78        | 37.46        | 36.83                       | 36.33        | 37.17        | 36.14        |
| DF           | 69.30        | 69.44        | 69.21        | 69.10        | 73.44         | 74.49        | 73.63        | 73.75        | 34.56                    | 36.71        | 34.56        | 34.56        | 34.58                       | 34.51        | 34.58        | 33.78        |
| FS-Net       | 85.75        | 85.91        | 85.46        | 85.56        | 84.50         | 84.82        | 84.91        | 84.77        | 45.24                    | 45.11        | 44.87        | 43.96        | 43.14                       | 46.42        | 43.33        | 43.48        |
| GraphDApp    | 86.53        | 84.18        | 85.66        | 85.96        | 80.32         | 81.21        | 80.93        | 80.47        | 41.41                    | 42.16        | 41.98        | 41.79        | 34.41                       | 34.32        | 34.26        | 34.24        |
| PERT         | 78.70        | 78.70        | 79.29        | 78.71        | 76.40         | 76.40        | 77.52        | 76.77        | 45.69                    | 45.69        | 49.20        | 46.18        | 45.96                       | 45.96        | 47.54        | 45.72        |
| ET-BERT      | 86.54        | 86.54        | 86.72        | 86.57        | 87.82         | 87.72        | 87.94        | 87.92        | 57.04                    | 57.04        | 59.37        | 56.71        | 57.08                       | 57.08        | 60.07        | 56.98        |
| <b>ETool</b> | <b>93.14</b> | <b>94.21</b> | <b>93.77</b> | <b>93.19</b> | <b>93.39</b>  | <b>92.66</b> | <b>91.83</b> | <b>92.11</b> | <b>74.38</b>             | <b>76.24</b> | <b>74.12</b> | <b>74.88</b> | <b>73.19</b>                | <b>74.24</b> | <b>72.18</b> | <b>72.13</b> |

as CUMUL and AppScanner), indicating the diminishing effectiveness of expert-driven feature extraction. DF and FS-Net, which employ deep learning for feature extraction, can recognize encrypted traffic with I.I.D. flows. However, their performance varies due to differences in network architecture and feature selection, and both are constrained by the limited size of labelled data. Although ET-BERT demonstrates stronger recognition performance, confirming the advantage of pre-training over traditional methods, its effectiveness is hindered by the limitations of flow-level input. In contrast, ETool does not rely on extensive flow pre-training; instead, it leverages the comprehension capabilities of large language models through instruction tuning, exploiting multi-flow correlations and contextual relationships from limited labelled data, thus enabling effective performance in challenging I.I.D. scenarios.

In the EAC-VERSION task, ETool significantly outperforms the three leading methods, with F1 score improvements of 4.19%, 7.34%, and 18.82%, respectively. The increased difficulty of identifying encrypted traffic stems from the larger number of data categories for recording application versions, as well as the interference of similar flows. Despite these challenges, ETool shows superior robustness, affirming its powerful capacity to comprehend and generalize traffic features effectively.

2) *Non-I.I.D. Encrypted Traffic Classification*: In this subsection, we explore the generalization ability of our model by incorporating more instruction data to fine-tune the ETool for effectively handling various types of tasks. In contrast to supervised I.I.D. experiments, this subsection will discuss the out-of-distribution generalisation capabilities of our proposed method under distributional variations.

According to Table IV, the baseline method shows a significant degradation in performance in the face of changes in flow distribution. Nevertheless, our proposed ETool achieves the lowest performance degradation and an average performance improvement of 16.66%, 29.79%, and 33.42% on F1 compared to different representative baselines in the APP53  $\Rightarrow$  TIME and APP53  $\Rightarrow$  VERSION tasks, respectively.

In the EAC  $\Rightarrow$  TIME task, the traffic identification accuracy of existing representative encrypted traffic classification

methods decreases significantly when no out-of-distribution handling strategies are applied. This decline is particularly evident when the methods are tested on new distributions. For instance, AppScanner achieves an F1 score of 72.27% on I.I.D. traffic, yet its performance drops to 44.03% when evaluated on test data with time shifts. Similarly, FS-Net and ET-BERT exhibit declines in F1 scores, dropping to 43.96% and 56.71%, respectively. These results illustrate that shifts in traffic distribution, caused by changes in time intervals, have a substantial impact on the performance of existing methods.

However, the ETool framework demonstrates superior robustness, experiencing minimal degradation and exhibiting enhanced O.O.D. traffic identification capabilities compared to the other methods. Unlike these methods, which struggle to adapt to O.O.D. traffic caused by time variation, ETool is capable of maintaining effective performance. This demonstrates that, despite shifts in the distribution of traffic over time, certain invariant properties persist in O.O.D. traffic. These properties, rooted in the associations within the traffic transport topology and the contextual relationships of traffic flows, can be effectively captured and integrated into ETool's inference mechanism, allowing it to generalize and perform well even under time-varying conditions.

In the EAC  $\Rightarrow$  VERSION task, we observe that the impact from version updates is relatively stronger than temporal changes, seeing that this is a more challenging task for identifying out-of-distribution encrypted traffic. Unlike time spanning, the cross-version task is to collect encrypted traffic from two different versions of a mobile application at the same time, when differences in application network interfaces or service design logic bring about differences in traffic distribution. AppScanner's traffic identification result drops to 35.00%, while FS-Net and ET-BERT both drop to 43.48% and 56.98% respectively. However, the ETool model still maintains minimal performance degradation and provides a significant improvement over existing methods.

Furthermore, we validate the performance of ETool in the detection of malicious traffic. When distinguishing between benign and malicious traffic, a key source of distributional bias arises from the variation among different botnet types present in the malicious traffic. In addition to the variation

TABLE V  
COMPARISON RESULTS ON ISCX-BOTNET IN O.O.D. SETTING

| Dataset      | MSC $\Rightarrow$ T (Binary) |              |              |              | MSC $\Rightarrow$ T (Multi-Class) |              |              |              |
|--------------|------------------------------|--------------|--------------|--------------|-----------------------------------|--------------|--------------|--------------|
| Method       | AC%                          | PR%          | RC%          | F1%          | AC%                               | PR%          | RC%          | F1%          |
| AppScanner   | 76.94                        | 76.97        | 76.95        | 76.92        | 60.99                             | 60.12        | 60.99        | 60.16        |
| CUMUL        | 65.95                        | 66.77        | 65.88        | 65.42        | 29.41                             | 56.96        | 29.41        | 28.37        |
| DF           | 78.25                        | 79.90        | 78.28        | 77.94        | 26.70                             | 44.30        | 42.10        | 42.30        |
| FS-Net       | 77.42                        | 78.81        | 77.36        | 77.27        | 70.33                             | 71.39        | 71.34        | 69.87        |
| GraphDApp    | 77.92                        | 78.13        | 77.43        | 77.81        | 71.36                             | 71.29        | 71.37        | 71.31        |
| PERT         | 85.23                        | 88.77        | 85.11        | 85.19        | 65.38                             | 62.70        | 65.36        | 61.48        |
| ET-BERT      | 85.74                        | 85.18        | 88.67        | 85.87        | 65.51                             | 72.58        | 65.51        | 67.91        |
| <b>ETool</b> | <b>95.36</b>                 | <b>95.72</b> | <b>94.87</b> | <b>95.03</b> | <b>84.19</b>                      | <b>83.02</b> | <b>82.73</b> | <b>81.95</b> |

factors encountered in the two previously mentioned O.O.D. tasks, the MSC  $\Rightarrow$  T (Binary) task introduces another layer of complexity by including malicious traffic from previously unknown botnet types. This necessitates a higher degree of model generalization, as the model must effectively generalize beyond the known botnet types to accurately detect and classify these unseen forms of malicious traffic. Based on the characteristics of the dataset, we also incorporated benign traffic alongside seven types of botnet traffic to perform a multi-class classification task for malicious traffic detection.

As shown in Table V, AppScanner and FS-Net, both of which rely on packet size as a feature, exhibit superior detection rates compared to their counterparts (excluding unknown botnet types). This suggests that packet size is a valuable feature for capturing commonalities across distributional shifts. Additionally, the use of datagrams as feature carriers in DF, compared to pre-training methods such as PERT and ET-BERT, further supports the performance gains attributable to the pre-training architecture. Notably, ETool surpasses the optimal baseline by 9.16% and 12.08% across two malicious traffic detection tasks. On the one hand, ETool leverages TRG to fuse packet size and datagram features, alongside incorporating concurrency and timing relationships of flows. This allows the model to capture richer underlying interactions within the traffic data. Second, ETool’s large-scale architecture enhances its inference capabilities through pre-training, while also improving its understanding of graph structures. These factors collectively enable ETool to generalize more effectively to unseen traffic distributions.

### C. Ablation Study (RQ2)

We conduct an ablation study to investigate the individual contributions of different sub-modules of our proposed framework, and the results are reported in Table VI.

We sequentially eliminate raw datagram, packet length, graph structural tuning, and LLM and show the ablation results to verify the contribution of each component on different tasks.

- (1) In Models “1-2”, we evaluate the impact of different granularities of traffic input information on the model’s effectiveness. Model “1”, which excludes datagram sequences, shows an average decrease of 4.02%

in F1 score compared to ETool. Similarly, removing packet length sequences results in an average decrease of 2.44%. These findings suggest that while the representation of arbitrary traffic information provides some representational gain, datagrams offer more significant improvements in model performance.

- (2) In Model “3”, we assess the impact of the traffic graph structure and the graph instruction fine-tuning task. In this model, flow-level instruction fine-tuning is performed directly using the large language model, without incorporating traffic relation graphs. Model “3” exhibits significant performance degradation across all scenarios, with an average reduction of 22.13% in F1 score compared to the full model. These results suggest that both the traffic correlation structure and the graph instruction tuning paradigm are critical for enabling ETool to learn traffic context more effectively. Furthermore, this interaction-based approach enhances the model’s ability to capture representational similarities under distributional shifts, thereby improving its performance in O.O.D. traffic detection.
- (3) Model “4” was designed to perform supervised training on flow graphs using an uninitialized Graph Transformer model, removing the instruction tuning paradigm, and then testing its out-of-distribution capability. Compared to ETool, Model “4” demonstrates an average F1 score decrease of 12.84% across all datasets. This indicates that the understanding and reasoning capabilities provided by the large language model play a critical role in mitigating misclassification after distributional shifts, offering substantial support for the task of O.O.D. traffic identification.

### D. Generalization Ability Investigation (RQ3)

To further measure the performance difference between the ETool model and the comparative approach models, we analyze the encrypted traffic classification capability under the ISCX-VPN with I.I.D. setups and the dynamic distribution-variation traffic dataset NETD.

As described in Section VIII-A1, we further illustrate the detailed construction process. NETD is primarily designed by controlling two key factors: proportional bias and compositional bias. The specific settings are as follows:

**Basic Dataset Composition:** The ISCX-VPN dataset consists of 6 types of services under both VPN and Non-VPN categories, encompassing a total of 17 applications, namely Chat (ICQ, AIM, Skype, Facebook and Hangouts), Email (SMTPS, POP3S and IMAPS), File Transfer (Skype, FTPS and SFTP), P2P (uTorrent and Transmission), Streaming (Vimeo and Youtube), VoIP (Facebook, Skype and Hangouts). With the detection objective of service traffic identification in mixed traffic scenario, the components of the constituent services are applied.

**Proportional Bias Setting:** In this setting, we ensure that the constituent components of each target class are present in both the training and testing data. For each service category, one primary component is randomly selected. The proportional bias between the primary and other components is controlled



TABLE VI  
ABLATION STUDY OF KEY COMPONENTS IN ETool

| Dataset            |                             | APP53-T      |              | APP53-V      |              | APP53 $\Rightarrow$ T |              | APP53 $\Rightarrow$ V |              | Botnet $\Rightarrow$ T |              |
|--------------------|-----------------------------|--------------|--------------|--------------|--------------|-----------------------|--------------|-----------------------|--------------|------------------------|--------------|
| Method             |                             | AC%          | F1%          | AC%          | F1%          | AC%                   | F1%          | AC%                   | F1%          | AC%                    | F1%          |
| ETool (Full Model) |                             | <b>93.14</b> | <b>93.19</b> | <b>93.39</b> | <b>92.11</b> | <b>74.38</b>          | <b>74.88</b> | <b>73.19</b>          | <b>72.13</b> | <b>95.36</b>           | <b>95.03</b> |
| 1                  | w/o Raw Datagram            | 90.93        | 89.62        | 89.33        | 88.64        | 68.76                 | 68.18        | 67.26                 | 67.93        | 93.26                  | 92.83        |
| 2                  | w/o Packet Length           | 91.73        | 90.62        | 91.31        | 90.99        | 69.43                 | 69.39        | 70.55                 | 69.72        | 94.75                  | 94.39        |
| 3                  | w/o Graph Structural Tuning | 74.25        | 73.92        | 76.74        | 76.81        | 45.87                 | 45.79        | 43.56                 | 43.11        | 77.83                  | 77.02        |
| 4                  | w/o Large Language Model    | 87.26        | 86.95        | 87.01        | 85.72        | 55.61                 | 54.79        | 52.20                 | 51.96        | 83.47                  | 83.69        |

by specifying a dominant ratio, which determines the relative prevalence of the primary component:

$$\text{Dominant Ratio} = \frac{N_{\text{Dominant}}}{N_{\text{Minor}}} \quad (8)$$

where  $N_{\text{Dominant}}$  denotes the number of samples for the dominant component, while  $N_{\text{Minor}}$  represents the average number of samples for the minor components. By fixing the dominant ratio in either the training or testing data and varying the proportional bias in the other, we simulate different distribution shift scenarios.

**Compositional Bias Setting:** In contrast to proportional bias, compositional bias simulates the situation where knowledge in the training data fails to cover the complete distribution. By varying the number of constituent components for each service category in the training and test data, we are able to simulate different degrees of information loss and thus achieve distributional bias. For the set of contextual components  $C'$ , the construction strategy for the training and testing set is as follows:

$$\mathcal{T} = \left\{ T \subset C' \mid 1 \leq |T| \leq N-1 \right\},$$

$$|\mathcal{T}| = \sum_{k=1}^{N-1} \binom{N}{k} = 2^N - 2 \quad (9)$$

$$\mathcal{S} = \left\{ S \subset C' \mid 1 \leq |S| \leq N \right\},$$

$$|\mathcal{S}| = \sum_{m=1}^N \binom{N}{m} = 2^N - 1 \quad (10)$$

where  $\mathcal{T}$  denotes the optional set of training data,  $\mathcal{S}$  denotes the optional set of testing data, and  $N$  denotes the full number of contextual components of the current category.

On the basis of ISCX-VPN, we construct generate four O.O.D. traffic datasets with different distribution shifts: (1) NETD-1: The distribution of the test set and training set of traffic data is changed by using a proportional bias strategy, and the dataset is generated by randomly sampling according to the ratio of major and minor components of 1:3. (2) NETD-2: Similar to NETD-1, but randomly sampling according to a 3:1 ratio of major and minor components in the sample pool. (3) NETD-3: The distribution of the traffic data training set is changed by employing a contextual compositional component bias strategy and tested on the full data set. We randomly capture 80% of the applications in the

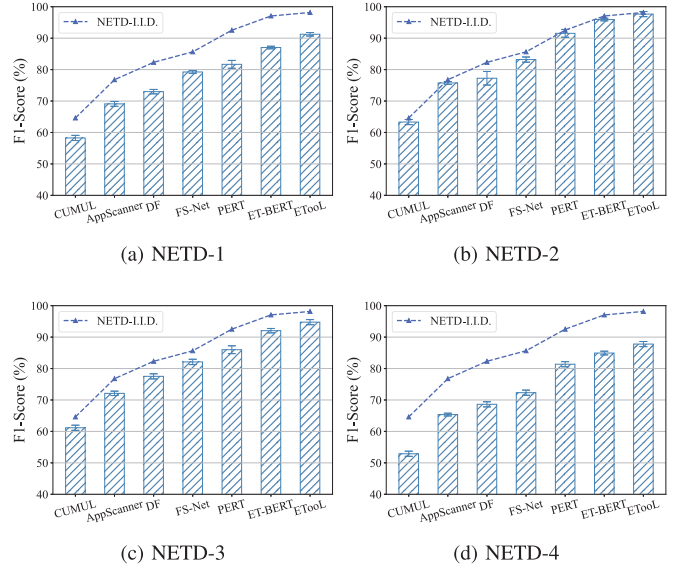


Fig. 4. Comparison results on dynamic Non-I.I.D. encrypted traffic dataset.

target class of services as contextual constituents, while other applications do not appear in the training set. It is also possible to further construct the training data with more severely shifted distributions according to the ratio bias of the major and minor components. (4) NETD-4: Similar to NETD-3, but captures 20% of the contextual applications of the target service.

According to the results in Fig. 4, the visualization clearly demonstrates that ETool achieves the best classification performance across various datasets. In the figure, the recognition results for each method are represented by bars, while the folded lines indicate performance under the I.I.D. setting. Notably, ETool and ET-BERT show comparable performance in the I.I.D. setting. However, under two distinct Non-I.I.D. scenarios, ETool significantly outperforms the other methods, demonstrating superior robustness and classification accuracy.

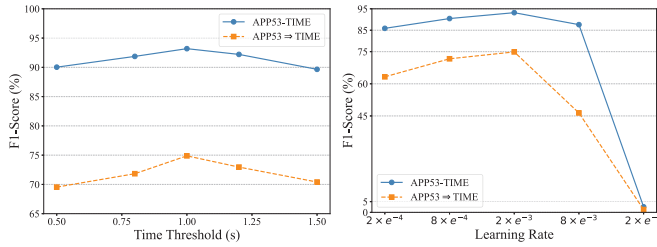
#### E. Model Efficiency Study (RQ4)

The study aims to evaluate the computational efficiency of our model during the training stage.

As shown in Table VII, our instruction tuning framework follows a two-stage process in which both LLM and graph encoder parameters are frozen and only the flow-graph

TABLE VII  
STUDY ON THE TIME AND SPACE EFFICIENCY OF THE MODEL TRAINING AND INFERENCE

|   | Fine-tuning Method          | Training Time | Tuned Parameters | GPU(MB) | FLOPs         | Inference Latency(ms) |
|---|-----------------------------|---------------|------------------|---------|---------------|-----------------------|
| 1 | BURST Graph Matching tuning | OOM           | 6,607,884,288    | OOM     | 6,704,861,184 | -                     |
| 2 | BURST Graph Matching freeze | 18:36:27      | 131,612,672      | 73,511  | 2,915,157,037 | 3,756.62              |
| 3 | Traffic Task tuning         | OOM           | 6,607,884,288    | OOM     | 6,704,861,184 | -                     |
| 4 | Traffic Task freeze         | 1:38:41       | 131,612,672      | 71,196  | 2,793,692,160 | 3,756.62              |



(a) Impact of BURST Time Threshold (b) Impact of Learning Rate

Fig. 5. Comparison results on different hyper-parameters selection.

aligned projection layer is tuned. We perform a comparison between freezing and tuning LLM parameters in a dual-card 80G Nvidia A800 environment, denoted by *freeze* and *tuning*, respectively. The study investigates the time and space efficiency in terms of training time, tuning parameters, single GPU memory occupied (MB), model computing volume and inference latency (milliseconds per response). Under the same experimental conditions, we suffer from GPU Out of Memory errors when fully parametrically tuning a large language model even with one batch size. However, by using a parameter-freezing tuning strategy, the training process can still be executed normally when increasing the training batch size. In addition, the parameters involved in instruction tuning is reduced by more than 50 times with frozen compared to full-parameter, resulting in a significant reduction in model computation and training time. To further investigate the inference efficiency of ETool, we measured the inference latency on the NETD dataset, using a single NVIDIA A800. While ETool has not yet met the requirements for real-time detection, it is well-suited for scenarios such as assisted decision-making, where accurate O.O.D. encrypted traffic identification is critical. In particular, when combined with interpretability strategies [50], LLM’s generalization capabilities can be regularized, making it suitable for online deployment.

#### F. Hyper-Parameters Analysis (RQ5)

The analysis aims to evaluate the selection of hyper-parameters during the training stage. According to the results presented in Figure 5, the BURST time threshold and the learning rate affect the performance of the model in the test scenarios of I.I.D. and O.O.D.

If the BURST time threshold is too small, flows serving different functionalities may be aggregated into the same

BURST, while an overly large threshold may also lead to the merging of flows with unrelated functions. To evaluate this effect, we experimented with a range of time thresholds and observed the testing performance. Empirically, a threshold of around 1 second yielded the most favorable results.

Moreover, due to the limitation on batch size, setting the learning rate too high can cause the model to oscillate or overshoot near the convergence point, leading to increased gradient variance and potential divergence. On the other hand, setting the learning rate too low slows down convergence, requiring more training steps to compensate. Through experiments across different learning rate ranges, we found that setting the learning rate to  $2 \times 10^{-3}$  yields the best performance.

## IX. CONCLUSION

In this paper, we propose ETool, an effective and distributionally adaptive traffic large language model, aiming to improve the generalisation ability of traffic classification model. The proposed framework injects traffic graph structures based on flow interaction knowledge into the LLM tuning paradigm. We comprehensively evaluate the generalisation ability of ETool on seven encrypted traffic datasets in I.I.D. and Non-I.I.D. settings, demonstrating the effectiveness of our approach in both supervised and zero-shot scenarios. The experimental results clearly demonstrate that our proposed method exhibits superior out-of-domain generalization capabilities compared to existing encrypted traffic classification approaches. The ETool framework effectively integrates traffic features and interaction correlation patterns with adaptive instruction tuning via large language models. This enables ETool to identify out-of-distribution traffic while retaining knowledge of traffic from previous distributions, offering a significant advantage over traditional models that rely on iterative retraining.

## ACKNOWLEDGMENT

The authors would like to express their grateful appreciation to the associate editor and the anonymous reviewers for their valuable efforts in greatly improving this article.

## REFERENCES

- [1] S. Rezaei and X. Liu, “Deep learning for encrypted traffic classification: An overview,” *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [2] C. Fu, Q. Li, M. Shen, and K. Xu, “Realtime robust malicious traffic detection via frequency domain analysis,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds., Nov. 2021, pp. 3431–3446.

- [3] S. Luo et al., "An in-depth study of microservice call graph and runtime performance," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3901–3914, Dec. 2022.
- [4] K. Ye, H. Shen, Y. Wang, and C.-Z. Xu, "Multi-tier workload consolidations in the cloud: Profiling, modeling and optimization," *IEEE Trans. Cloud Comput.*, vol. 10, no. 2, pp. 899–912, Apr. 2022.
- [5] Q. Yuan, G. Gou, Y. Zhu, G. Xiong, and Y. Wang, "MCRe: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 133–147, 2024.
- [6] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 1723–1732.
- [7] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [8] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Jul. 2018, pp. 1928–1943.
- [9] P. Liu, L. Li, Y. Yan, M. Fazzini, and J. Grundy, "Identifying and characterizing silently-evolved methods in the Android API," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, Madrid, Spain, May 2021, pp. 308–317.
- [10] E. Gourdin, P. Maillé, G. Simon, and B. Tuffin, "The economics of CDNs and their impact on service fairness," *IEEE Trans. Netw. Service Manage.*, vol. 14, no. 1, pp. 22–33, Mar. 2017.
- [11] M. Market. (2020). *Mobile CDN Market — Global Industry Report*. [Online]. Available: <https://www.transparencymarketresearch.com/mobile-cdn-market.html>
- [12] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized data-gram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Lyon, France, F. Laforest, R. Troncy, E. Simperl, D. Agarwal, A. Gionis, and I. Herman, Eds., 2022, pp. 633–642.
- [13] T. Cui et al., "TrafficLLM: Enhancing large language models for network traffic analysis with generic traffic representation," 2025, *arXiv:2504.04222*.
- [14] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Aug. 2018.
- [15] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Jul. 2016.
- [16] Y. Tian, R. Gan, Y. Song, J. Zhang, and Y. Zhang, "ChiMed-GPT: A Chinese medical large language model with full training regime and better alignment to human preferences," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, Bangkok, Thailand, 2024, pp. 7156–7173.
- [17] K. Yang, T. Zhang, Z. Kuang, Q. Xie, J. Huang, and S. Ananiadou, "MentalLLaMA: Interpretable mental health analysis on social media with large language models," in *Proc. ACM Web Conf.*, Singapore, May 2024, pp. 4489–4500.
- [18] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [19] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [20] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.
- [21] J. Wei et al., "Emergent abilities of large language models," in *Proc. Trans. Mach. Learn. Res.*, Aug. 2022.
- [22] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Paris, France, Jul. 2019, pp. 1171–1179.
- [23] C. Liu, W. Wang, M. Wang, F. Lv, and M. Konan, "An efficient instance selection algorithm to reconstruct training set for support vector machine," *Knowl.-Based Syst.*, vol. 116, pp. 58–73, Jan. 2017.
- [24] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Ha Noi, Vietnam, Dec. 2020, pp. 1–8.
- [25] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. CVPR*, Colorado Springs, CO, USA, Jun. 2011, pp. 1521–1528.
- [26] J. Tang et al., "GraphGPT: Graph instruction tuning for large language models," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Washington, DC, USA, Jul. 2024, pp. 491–500.
- [27] E. B. Beigi, H. H. Jazi, N. Stakhanova, and A. A. Ghorbani, "Towards effective feature selection in machine learning-based botnet detection approaches," in *Proc. IEEE Conf. Commun. Netw. Secur.*, San Francisco, CA, USA, Sep. 2014, pp. 247–255.
- [28] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, P. Mori, S. Furnell, Eds., Feb. 2017, pp. 253–262.
- [29] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, Rome, Italy, P. Mori, Ed., Feb. 2016, pp. 407–414.
- [30] X. Lin et al., "CETP: A novel semi-supervised framework based on contrastive pre-training for imbalanced encrypted traffic classification," *Comput. Secur.*, vol. 143, Aug. 2024, Art. no. 103892.
- [31] M. Jiang, M. Cui, C. Liu, G. Gou, G. Xiong, and Z. Li, "Zero-relabelling mobile-app identification over drifted encrypted network traffic," *Comput. Netw.*, vol. 228, Jun. 2023, Art. no. 109728.
- [32] R. Attarian, L. Abdi, and S. Hashemi, "AdaWFPA: Adaptive online website fingerprinting attack for tor anonymous network: A stream-wise paradigm," *Comput. Commun.*, vol. 148, pp. 74–85, Dec. 2019.
- [33] Q. Meng et al., "Beyond known threats: A novel strategy for isolating and detecting unknown malicious traffic," *J. Inf. Secur. Appl.*, vol. 89, Mar. 2025, Art. no. 103920.
- [34] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Jul. 2020.
- [35] Y. He, Z. Shen, and P. Cui, "Towards non-I.I.D. image classification: A dataset and baselines," *Pattern Recognit.*, vol. 110, Feb. 2021, Art. no. 107383.
- [36] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Netw.*, vol. 34, no. 4, pp. 20–27, Jul. 2020.
- [37] F. Petroni et al., "Language models as knowledge bases?," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, Jul. 2019, pp. 2463–2473.
- [38] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, "A critical evaluation of website fingerprinting attacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Scottsdale, AZ, USA, Nov. 2014, pp. 263–274.
- [39] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
- [40] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [41] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.
- [42] X. Wang et al., "Combine intra- and inter-flow: A multimodal encrypted traffic classification model driven by diverse features," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110403.
- [43] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Learning to prompt for vision-language models," *Int. J. Comput. Vis.*, vol. 130, no. 9, pp. 2337–2348, Sep. 2022.
- [44] J. He et al., "TransFG: A transformer architecture for fine-grained recognition," in *Proc. 36th AAAI Conf. Artif. Intell., 34th Conf. Innov. Appl. Artif. Intell.*, May 2022, pp. 852–860.
- [45] D. Wu et al., "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM Conf.*, Sydney, NSW, Australia, Aug. 2024, pp. 661–678.
- [46] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, A. Zhang and H. Rangwala, Eds., Aug. 2022, pp. 2576–2584.
- [47] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Hong Kong, Oct. 2023, pp. 297–314.
- [48] Y. Wang et al., "Self-instruct: Aligning language models with self-generated instructions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, Toronto, ON, Canada, Oct. 2023, pp. 13484–13508.

- [49] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, and J. Han, "Large language models on graphs: A comprehensive survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 8622–8642, Dec. 2024.
- [50] D. Han et al., "Rules refine the riddle: Global explanation for deep learning-based anomaly detection in security applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Salt Lake City, UT, USA, Dec. 2024, pp. 4509–4523.



**Wenqi Dong** received the B.E. degree from Qingdao University, China, in 2022. He is currently pursuing the Ph.D. degree with the Institute of Information Engineering, University of Chinese Academy of Sciences, Beijing. His research interests include cyber security and network traffic analysis.



**Xinjie Lin** received the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, and the School of Cyber Security, University of Chinese Academy of Sciences, China, in 2024. He is currently an Assistant Researcher with Zhongguancun Laboratory, Beijing, China. His research interests include networks and artificial intelligence security.



**Jing Yu** received the B.S. degree in automation science from Minzu University of China in 2011, the M.S. degree in pattern recognition from Beihang University, China, in 2014, and the Ph.D. degree from the University of Chinese Academy of Sciences, China, in 2019. She is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. Her research interests include cross-modal understanding.



**Gang Xiong** (Member, IEEE) is currently a Full Professor and a Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. He has authored more than 110 papers in refereed journals and conference proceedings. His research interests include cyber security, network measurement, network traffic analysis, and network forensics.



**Zhen Li** received the B.Eng. degree from Shandong University, China, in 2009, the M.S. degree from the Institute of Computing Technology, Chinese Academy of Sciences, China, in 2012, and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, in 2020. His research interests include encrypted network behavior analysis and network resource measurement.



**Gaopeng Gou** received the B.E., M.Eng., and Ph.D. degrees from Beihang University, China, in 2005, 2008, and 2014, respectively. He is currently a Senior Engineer and a Full Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. His research interests include network security and anomaly detection.



**Wei Xia** received the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2022. She is currently a Senior Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include network measurement and behavior analysis.